Labo — Détection avec Suricata

Nov 4, 2026 · François Paquin, CISSP

Vous transformez les constats de l'exercice Angry Poutine en règles Suricata, puis vous surveillez le trafic rejoué en direct dans EveBox.

Élément	Détails
Durée prévue	Environ 2 h 45
Environnement	Votre VM Ubuntu 26.04.1 du Cyber Range, avec accès Internet pour l'installation
Fichiers fournis	Dossier `Suricata` du dépôt des labos et capture de l'exercice Angry Poutine
À avoir sous la main	Votre rapport d'incident et vos IOC d'Angry Poutine
Remise	Un rapport (Word ou PDF) avec vos réponses, votre fichier `labo.rules` et le tableau de synthèse

Objectifs

- Installer Suricata sans plateforme complète et lire ses journaux `fast.log` et `eve.json`.
- Écrire des règles à partir d'indicateurs de compromission : domaine, URL, IP, certificat et empreinte.
- Écrire des règles fondées sur un comportement et réduire leurs faux positifs.
- Rejouer une capture en direct avec tcpreplay et trier les alertes dans EveBox.
- Comparer la solidité des détections et justifier vos choix.

Règles du labo

- La capture contient un vrai maliciel Windows. Travaillez uniquement dans votre VM Ubuntu du Cyber Range.
- Ne copiez jamais la capture ni un fichier extrait vers un poste Windows ou personnel.
- Les adresses IP et les domaines de l'exercice sont réels et malveillants : ne les visitez pas.
- Toutes les heures sont à donner en UTC.
- Pour chaque réponse, notez la commande ou la règle qui la prouve.

Partie 1 — Installer et vérifier (25 min)

Cette partie se termine quand `analyser.sh` affiche 48 alertes pour la règle de test.

1.1 Installer les outils

```
sudo apt update
sudo apt install -y suricata suricata-update tcpreplay jq git unzip curl
```

```
sudo systemctl disable --now suricata
suricata -V
```
La troisième commande arrête le service installé par défaut : dans ce labo, vous lancez Suricata vous-même.

1.2 Récupérer le matériel

```
git clone --depth 1 https://github.com/SnailLabs-dev/Labs.git ~/Labs
cp -r ~/Labs/Suricata ~/labo
cd ~/labo
chmod +x analyser.sh
unzip -P infected_20210910 "$HOME/Labs/Exercice Angry Poutine/2021-09-10-traffic-analysis-exercise.pcap.zip"
mv 2021-09-10-traffic-analysis-exercise.pcap angry-poutine.pcap
sha256sum angry-poutine.pcap
```
L'empreinte attendue commence par `a9576008f7fd` et se termine par `b2937a205cd214cdde09f6924f0921b8e266dd12`.

1.3 Lancer la première analyse

```
./analyser.sh
```
Le script lit la capture avec le fichier `regles/labo.rules`, qui contient une seule règle de test. Il écrit ses résultats dans le dossier `sortie`.

Questions

1. Ouvrez `analyser.sh`. Expliquez le rôle des options `-c`, `--include`, `-S`, `-r`, `-k none` et `-l`.
2. Ouvrez `labo.yaml`. Pourquoi `HOME_NET` doit-il correspondre au réseau de la capture plutôt qu'à celui de votre VM ?
3. Pourquoi `-k none` est-il utile quand on analyse une capture ?

Partie 2 — Lire ce que Suricata produit (25 min)

Suricata n'a pas besoin d'alertes pour être utile : `eve.json` décrit déjà une grande partie du trafic. Commencez par `fast.log`, le journal d'alertes le plus simple.

```
head -3 sortie/fast.log
jq -r .event_type sortie/eve.json | sort | uniq -c | sort -rn
```
Filtrez ensuite `eve.json` par type d'événement. Voici trois exemples à adapter :

```
# Noms demandés au DNS
jq -r 'select(.event_type=="dns" and .dns.type=="request") | .dns.queries[0].rrname' sortie/eve.json | sort -u

# Comptes vus dans Kerberos
jq -r 'select(.event_type=="krb5") | .krb5.cname' sortie/eve.json | sort | uniq -c

# Connexions TLS par SNI
jq -r 'select(.event_type=="tls") | .tls.sni // "(aucun SNI)"' sortie/eve.json | sort | uniq -c | sort -rn
```

Questions

1. Quels types d'événements trouvez-vous dans `eve.json` ? Pourquoi dit-on que Suricata est plus qu'un IDS ?
2. Le compte `hobart.gunnarsson` apparaissait-il dans vos flux NetFlow ? Quelle ligne de commande le retrouve ici ?
3. Combien de connexions TLS n'ont aucun SNI ? Vers quelles adresses vont-elles ?
4. Trouvez le `flow_id` du téléchargement depuis `simpsonsavingss.com`, puis affichez tous les événements qui portent ce numéro. Quel est l'intérêt de ce champ pour un analyste ?
5. Dans votre rapport Angry Poutine, quelles informations retrouvez-vous dans `eve.json` sans écrire une seule règle ?

Partie 3 — Écrire des règles à partir des IOC (50 min)

Chaque règle doit déclencher sur la capture, et le nombre d'alertes doit être expliqué. Ajoutez vos règles à `regles/labo.rules` et relancez `./analyser.sh` après chaque modification.

Une règle se lit ainsi :

```
alert dns $HOME_NET any -> any any (msg:"Description"; flow:to_server; dns.query; content:"exemple.com"; nocase; endswith; sid:1000001; rev:1;)
```

Partie	Rôle
`alert`	Action à prendre
`dns`	Protocole détecté, même hors de son port habituel
`$HOME_NET any -> any any`	Sources, ports et sens du trafic
`dns.query`	Tampon inspecté : le contenu qui suit s'applique à ce champ
`content`, `nocase`, `endswith`	Motif recherché et façon de le comparer
`sid`, `rev`	Identifiant unique de la règle et version

La documentation des règles décrit tous les mots-clés.

3.1 Le domaine

Écrivez une règle qui détecte la résolution du domaine de téléchargement. Combien d'alertes obtenez-vous ? Une seule infection peut-elle expliquer ce nombre ? Vérifiez les paquets avant de conclure.

3.2 L'URL

Écrivez une règle `http` qui combine `http.host` et `http.uri`. Pourquoi une règle basée seulement sur le nom `date1` serait-elle une mauvaise idée ?

3.3 Les adresses IP

Écrivez une seule règle `alert ip` pour les trois adresses malveillantes contactées par le poste. La syntaxe de liste est `[a.b.c.d,e.f.g.h]`.

- Combien d'alertes obtenez-vous ?
- Comparez ce nombre au nombre de sessions de votre analyse NetFlow.
- Pourquoi n'obtenez-vous pas une alerte pour chaque paquet ?

3.4 Le certificat

Écrivez une règle `tls` avec `tls.cert_subject` qui détecte le certificat auto-signé de l'attaquant. Le serveur envoie le certificat, ce qui détermine le sens de la règle.

Votre règle ne déclenche pas sur chaque session TLS vers les deux serveurs. Expliquez pourquoi avec les événements `tls` de `eve.json`.

3.5 L'empreinte du fichier

Créez la liste d'empreintes :

```
echo eed363fc4af7a9070d69340592dcab7c78db4f90710357de29e3b624aa957cf8 >
regles/mauvais.sha256
```

Ajoutez ensuite une règle `alert http any any -> any any` qui utilise `filesha256:mauvais.sha256;`. Elle ne déclenchera pas au début, et ce n'est pas une erreur de syntaxe.

1. Affichez l'événement `fileinfo` du fichier. Comparez sa taille et son état à ce que vous savez du fichier.
2. Trouvez le réglage responsable : `grep -n 'response-body-limit' /etc/suricata/suricata.yaml`.
3. Retirez le `#` de la dernière ligne de `labo.yaml`, puis relancez l'analyse.
4. Expliquez ce que ce réglage a changé. Quel compromis un capteur en production doit-il faire ?

Partie 4 — Détecter sans connaître l'IOC (30 min)

Un attaquant change facilement de domaine et d'adresse IP. Ces règles cherchent plutôt une façon d'agir, au prix de faux positifs à maîtriser.

4.1 Un exécutable reçu en HTTP

Écrivez une règle `http` dans le sens du serveur vers le poste. Dans `file.data`, cherchez `MZ` au tout début du fichier (`depth:2`), puis la chaîne `This program cannot be run in DOS mode` tout près (`distance:0; within:128`).

Vous obtiendrez deux alertes. Identifiez le faux positif.

Pour l'exclure, il faut relier la requête et la réponse. Une même règle ne peut pas combiner `http.host`, vu dans la requête, et `file.data`, vu dans la réponse. Suricata refuse cette règle en signalant des directions contradictoires.

Utilisez plutôt deux règles liées par un `flowbit` :

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"Marque Windows Update";
flow:established,to_server; http.host; content:"windowsupdate.com"; endswith;
flowbits:set,labo.windowsupdate; flowbits:noalert; sid:1000006; rev:1;)
```

Ajoutez `flowbits:isnotset,labo.windowsupdate;` à votre règle. Quel est le risque d'une exclusion basée sur ce nom ?

4.2 Du TLS sans SNI

Écrivez une règle avec `tls.sni; absent;`. Sur cette capture, elle donne exactement les connexions suspectes. Pourquoi faut-il rester prudent sur un vrai réseau ?

Ajoutez ensuite ce seuil pour alerter seulement sur la répétition vers une même adresse :

```
threshold:type both, track by_dst, count 5, seconds 3600;
```

Combien d'alertes reste-t-il ? À quelle heure arrive la première, par rapport au début du beaconing ?

4.3 L'énumération du domaine

Écrivez une règle `smb` avec `dcerpc.iface:12345778-1234-abcd-ef00-0123456789ac;`, l'interface SAMR, dans le sens du poste vers le serveur.

1. Regroupez les alertes par minute. Pourquoi cette règle est-elle trop bruyante ?
2. Affinez-la avec `dcerpc.stub_data` pour chercher `domain admins`. Windows encode ce texte en UTF-16 : chaque lettre est suivie de l'octet nul, par exemple `d|00|o|00|m|00|`.

4.4 La vérification de l'IP publique

Écrivez une règle `tls` qui détecte `myexternalip.com` dans le SNI. Pourquoi est-ce un indicateur de comportement plutôt qu'un service à bloquer sans analyse ?

Partie 5 — Surveiller en direct avec EveBox (25 min)

Vous passez maintenant de l'analyse d'un fichier à la surveillance d'une interface. Le rejeu reste enfermé dans votre VM grâce à une paire d'interfaces virtuelles : ce qui entre dans `veth0` sort par `veth1`.

5.1 Préparer le rejeu

```
sudo ip link add veth0 type veth peer name veth1
sudo ip link set veth0 up
sudo ip link set veth1 up
curl -sSf https://evebox.org/install.sh | sh
evebox version
mkdir -p ~/labo/direct ~/labo/evebox
```

Si `evebox` est introuvable, ouvrez un nouveau terminal et réessayez. Après un redémarrage de la VM, recréez les interfaces.

5.2 Lancer les trois terminaux

Dans le **terminal 1**, démarrez EveBox :

```
evebox server --sqlite --no-auth --no-tls --input ~/labo/direct/eve.json -D
~/labo/evebox
```

Ouvrez Firefox dans la VM à l'adresse `http://127.0.0.1:5636`.

Dans le **terminal 2**, démarrez Suricata sur `veth1` :

```
sudo TZ=UTC suricata -c /etc/suricata/suricata.yaml --include ~/labo/labo.yaml --set default-rule-path=$HOME/labo/regles -S ~/labo/regles/labo.rules --af-packet=veth1 -k none -l ~/labo/direct
```
Attendez que Suricata indique que le moteur a démarré.

Dans le **terminal 3**, rejouez la capture 60 fois plus vite que la réalité :

```
cd ~/labo
sudo tcpreplay -i veth0 --multiplier=60 angry-poutine.pcap
```
Les 77 minutes de la capture durent ainsi environ 77 secondes. Cliquez sur **Refresh** dans EveBox pendant le rejeu.

5.3 Trier les alertes

1. Comment EveBox regroupe-t-il les alertes dans la boîte **Inbox** ? Que représente la colonne **#** ?
2. Quelle est la différence entre **Archive** et l'étoile qui escalade une alerte ?
3. Ouvrez l'alerte sur l'empreinte SHA256. Quelles informations sur le flux, la requête HTTP et le fichier trouvez-vous dans une seule vue ?
4. Triez les alertes comme un analyste : archivez le bruit et escaladez ce qui exige une action. Justifiez deux décisions.

5.4 Comparer le direct et le fichier

1. Quelle heure portent les alertes dans EveBox ? Pourquoi n'est-ce pas l'heure de l'incident ?
2. Faites le total des alertes dans `~/labo/direct/eve.json` et comparez-le à `sortie/eve.json`. Une règle déclenche moins souvent en direct. Laquelle, et pourquoi ?
3. Arrêtez Suricata avec Ctrl+C. Relancez-le avec `-l ~/labo/direct2` après avoir créé ce dossier, puis rejouez la capture avec `--topspeed` au lieu de `--multiplier=60`.
4. Comparez les paquets perdus (`drops`) affichés à l'arrêt de Suricata et le nombre d'alertes. Que retenez-vous pour le dimensionnement d'un capteur ?

Partie 6 — Synthèse (10 min)

Votre conclusion doit dire quelles règles vous garderiez en production, et pourquoi.

Complétez ce tableau pour au moins six de vos règles.

Règle (sid)	Ce qu'elle détecte	Alertes	Facile à contourner ?	Risque de faux positifs	À garder en production ?

Répondez ensuite en quelques lignes :

1. L'attaquant relance sa campagne demain avec un nouveau domaine, de nouvelles adresses IP et un fichier recompilé. Quelles règles déclenchent encore ?
2. Quelle règle aurait donné l'alerte la plus utile, et à quelle heure ? Combien de temps l'attaquant aurait-il eu avant son énumération du domaine ?
3. Quelles informations de votre rapport Angry Poutine Suricata ne peut-il pas vous donner ?

Bonus

Une liste d'IOC avec un dataset

Remplacez vos listes dans les règles par des fichiers, plus simples à tenir à jour.

```
printf '94.158.245.52\n167.172.37.9\n194.62.42.206\n' > regles/ioc-ip.lst
printf 'simpsonsavingss.com' | base64 > regles/ioc-dom.lst
```

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Liste IOC - IP"; flags:S,12;
ip.dst; dataset:isset,ioc-ip,type ipv4,load ioc-ip.lst; sid:1000011; rev:1;)
alert dns $HOME_NET any -> any any (msg:"Liste IOC - domaine"; flow:to_server;
dns.query; dataset:isset,ioc-dom,type string,load ioc-dom.lst; sid:1000012; rev:1;)
```

Pourquoi les domaines sont-ils encodés en base64 ? Pourquoi la règle IP donne-t-elle deux alertes de plus que votre règle `alert ip`, malgré la même liste ?

Les règles publiques ET Open

```
sudo suricata-update
```

Lancez la capture avec `/var/lib/suricata/rules/suricata.rules` plutôt qu'avec vos règles. Qu'est-ce qui déclenche ? Que vous apportent ces règles par rapport aux vôtres, et quel travail ajoutent-elles ?